

Compte Rendu de TP 1-2 : Analyse du trafic réseau avec Wireshark

1. Introduction

Rôle de Wireshark

Wireshark est un analyseur de protocoles réseau (sniffer) qui permet de capturer et d'examiner en détail le trafic circulant sur une interface réseau. Il traduit les flux binaires en paquets lisibles par l'humain.

Importance de l'analyse réseau

L'analyse réseau est cruciale car elle permet de passer d'une vision théorique (taper une commande) à une vision réelle (voir ce qui transite sur le câble). Pour un analyste en SOC (Security Operations Center), c'est l'outil de base pour identifier des comportements suspects, des tentatives d'intrusion ou de l'exfiltration de données.

1. Description des manipulations

L'objectif était d'observer l'empreinte réseau d'un scan de ports effectué avec l'outil Nmap.

- **Préparation** : Utilisation d'une machine Kali Linux (attaquant) et d'une machine Ubuntu (cible) sur un réseau Host-Only.
- **Étape 1** : Lancement de Wireshark sur l'interface active (`eth0` ou `enp0s3`) avec les privilèges `sudo`.
- **Étape 2** : Lancement du scan depuis le terminal Kali avec la commande : `nmap -sS 192.168.56.10`

- **Étape 3 :** Analyse de la capture sous Wireshark en utilisant des filtres spécifiques pour isoler le trafic pertinent.

2. Résultats obtenus

(Note : Insérez ici vos captures d'écran réelles issues des manipulations effectuées.)

Partie 1 lancement de wireshark et capture du trafic

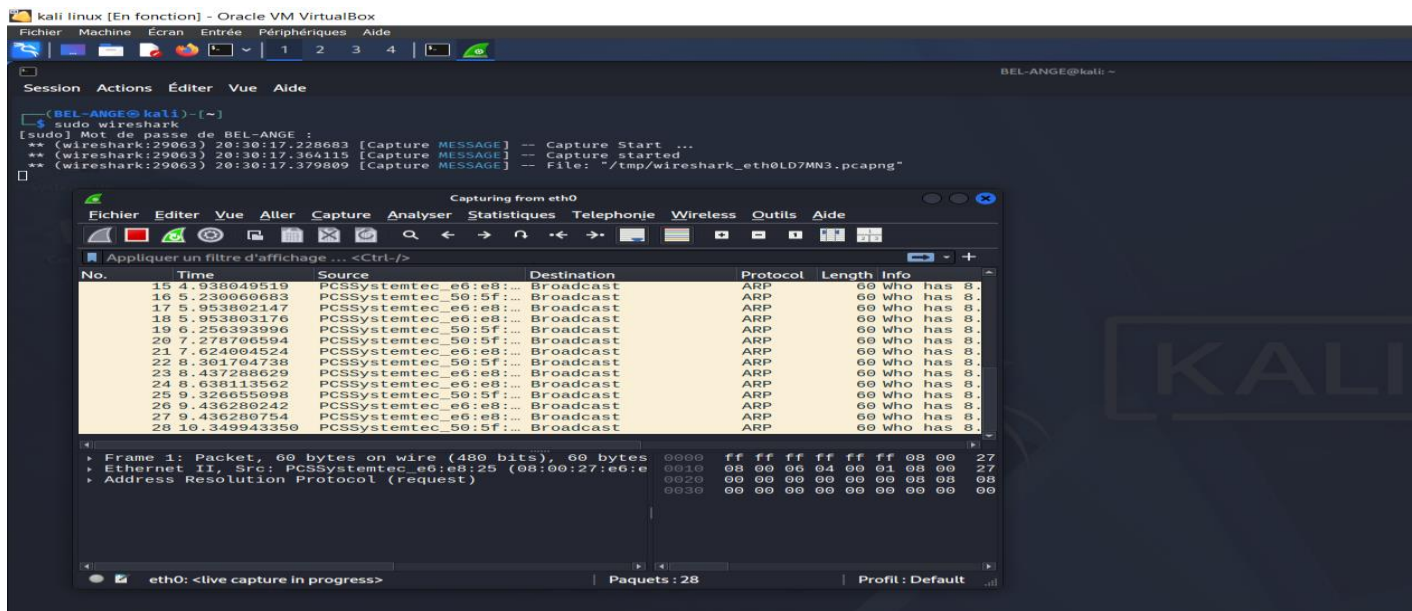
La commande utilisée pour generer ce trafic de scan est la suivante

Nmap -Ss 192.168.56.10

Nmap : l'outil de scan reseau

-Ss demande un tcp syn scan .cest ce parametre qui envoie les paquet

192.168.56.10 :ladresse ip de la machine cible(ubuntu)

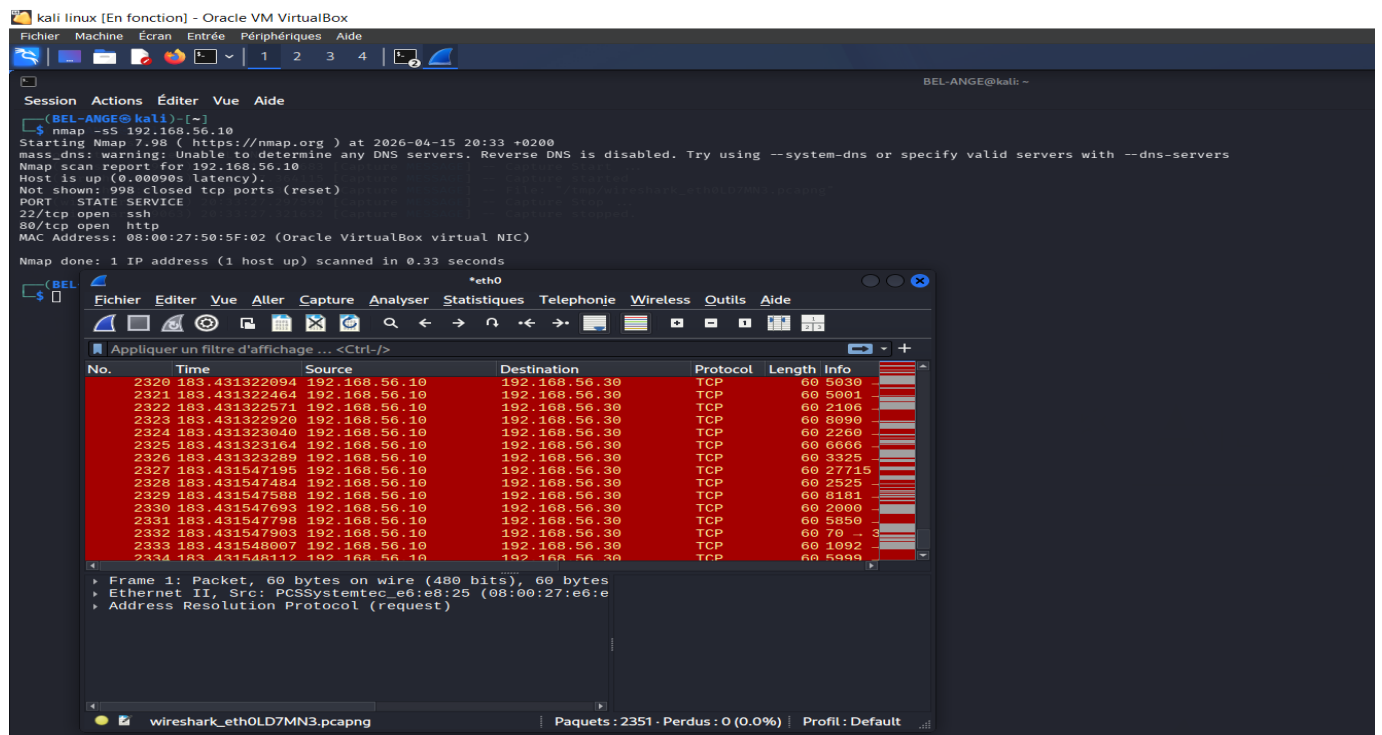


- Partie 2 generation dun trafic de scan

La commande qui a été executer pour generer cela est

Nmap – ss 192.168.56.10

Nous pouvons lire que le nmap a trouver deux ports ouvert sur la cible le port 22(ssh)et le port 80(http)

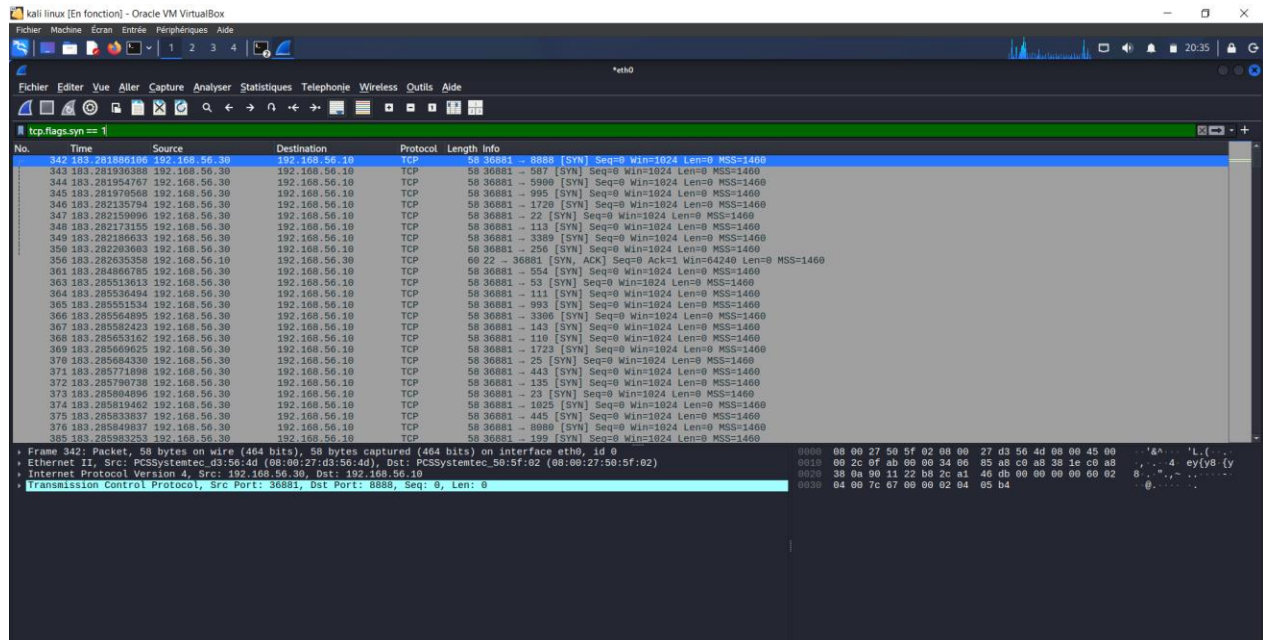


- Partie 3 filtrage des paquets

Le trafic afficher a été generer par commande suivante

Nmap –ss 192.168.56.10

On observe dans la colonne info une suite de port differents (8888,587,995,1723,...) soliciyees par la source 192.168.56.30.

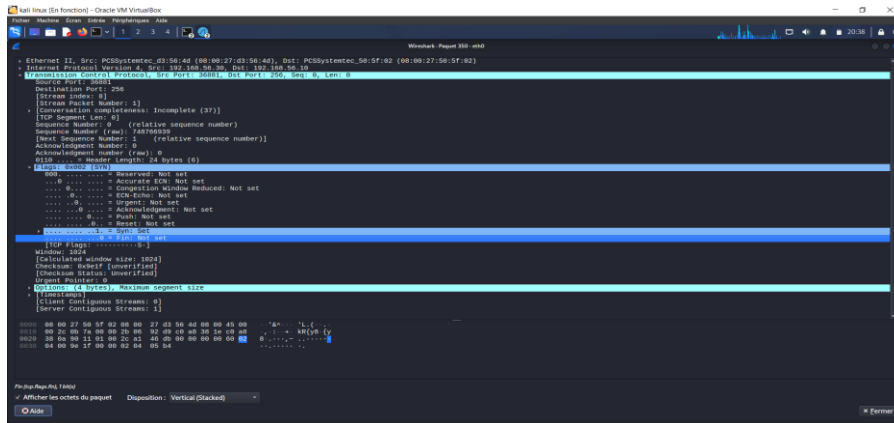


- Partie 4 analyse du fonctionnement tcp(syn/syn-ack)

La commande a lorigine de ce paquet est toujours le scan nmap lance precedemment

Nmap -ss 192.168.56.10

Il montre les details du protocole TCP pour un paquet specifique



- Partie 5 identification dun comportement malveillant

La commande utiliser est

Nmap –ss 192.168.56.10.

3. Analyse technique

Qu'est-ce qu'un paquet SYN ?

C'est le premier paquet envoyé pour initier la "triple poignée de main" (3-way handshake) du protocole TCP. Il signifie "Synchronize".

Comment détecter un port ouvert vs fermé ?

D'après l'analyse des flags TCP :

- **Cas 1 - Port Ouvert :** La cible répond par un **SYN-ACK** (Synchronize-Acknowledgment). L'attaquant sait que le port écoute.
- **Cas 2 - Port Fermé :** La cible répond par un **RST** (Reset). Cela indique que la connexion est refusée.

Reconnaître un scan réseau

Un scan se reconnaît par :

1. **La répétitivité** : Le même type de paquet (SYN) envoyé en boucle.
2. **La rapidité** : Des centaines de requêtes par seconde.
3. **La structure** : Un seul émetteur interrogeant une multitude de ports différents sur une courte période.

4. Utilisation avancée de Wireshark

Filtre Avancé	Utilité	Exemple concret
<code>ip.addr == 192.168.56.10</code>	Isole tout le trafic entrant ou sortant d'une IP spécifique.	Analyser uniquement les échanges avec la victime.
<code>tcp.port == 80</code>	Filtre le trafic sur un port spécifique (ici HTTP).	Vérifier si un serveur web est sollicité.
<code>http.request.method == "POST"</code>	Affiche uniquement les envois de données (formulaires, logins).	Détecter une tentative de brute-force.

-ip.addr==

kali linux [En fonction] - Oracle VM VirtualBox

Fichier Machine Écran Entrée Périphériques Aide

eth0

Fichier Édition Vue Aller Capture Analyser Statistiques Téléphonie Wireless Outils Aide

ip.addr == 192.168.56.10

No.	Time	Source	Destination	Protocol	Length	Info
342	183.281886106	192.168.56.30	192.168.56.10	TCP	58	36881 → 8888 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
343	183.281930388	192.168.56.30	192.168.56.10	TCP	58	36881 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
344	183.281954767	192.168.56.30	192.168.56.10	TCP	58	36881 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
345	183.281970568	192.168.56.30	192.168.56.10	TCP	58	36881 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
346	183.282135794	192.168.56.30	192.168.56.10	TCP	58	36881 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
347	183.282159096	192.168.56.30	192.168.56.10	TCP	58	36881 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
348	183.282173155	192.168.56.30	192.168.56.10	TCP	58	36881 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
349	183.282186633	192.168.56.30	192.168.56.10	TCP	58	36881 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
350	183.282203693	192.168.56.30	192.168.56.10	TCP	58	36881 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
351	183.282358229	192.168.56.10	192.168.56.30	TCP	60	8888 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
352	183.282388032	192.168.56.10	192.168.56.30	TCP	60	587 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
353	183.282388645	192.168.56.10	192.168.56.30	TCP	60	5900 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
354	183.282358733	192.168.56.10	192.168.56.30	TCP	60	995 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
355	183.282635114	192.168.56.10	192.168.56.30	TCP	60	1720 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
356	183.282635049	192.168.56.10	192.168.56.30	TCP	60	22 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
357	183.282635460	192.168.56.10	192.168.56.30	TCP	60	113 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
358	183.282635548	192.168.56.10	192.168.56.30	TCP	60	3389 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
359	183.282635606	192.168.56.10	192.168.56.30	TCP	60	256 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
360	183.282657716	192.168.56.30	192.168.56.10	TCP	54	36881 → 22 [RST] Seq=1 Win=0 Len=0
361	183.284866785	192.168.56.30	192.168.56.10	TCP	58	36881 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
362	183.285426614	192.168.56.10	192.168.56.30	TCP	60	554 → 36881 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
363	183.285513013	192.168.56.30	192.168.56.10	TCP	58	36881 → 54 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
364	183.285536494	192.168.56.30	192.168.56.10	TCP	58	36881 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
365	183.285551534	192.168.56.30	192.168.56.10	TCP	58	36881 → 993 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
366	183.285564895	192.168.56.30	192.168.56.10	TCP	58	36881 → 3386 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
367	183.285582423	192.168.56.30	192.168.56.10	TCP	58	36881 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Frame 350: Packet, 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface eth0, id 0

Ethernet II, Src: PCSysntec d3:56:dd (08:00:27:d3:56:dd), Dst: PCSysntec 58:5f:02 (08:00:27:50:5f:02)

Internet Protocol Version 4, Src: 192.168.56.30, Dst: 192.168.56.10

Transmission Control Protocol, Src Port: 36881, Dst Port: 256, Seq: 0, Len: 0

Source Port: 36881

Destination Port: 256

[Stream Index: 8]

[Stream Packet Number: 1]

[Conversation completeness: Incomplete (37)]

[TCP Segment Len: 0]

Sequence Number: 0 (relative sequence number)

Sequence Number (raw): 748706939

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 0

Acknowledgment number (raw): 0

010 ... 5 Header Length: 24 bytes (6)

Flags: 0x002 (SYN)

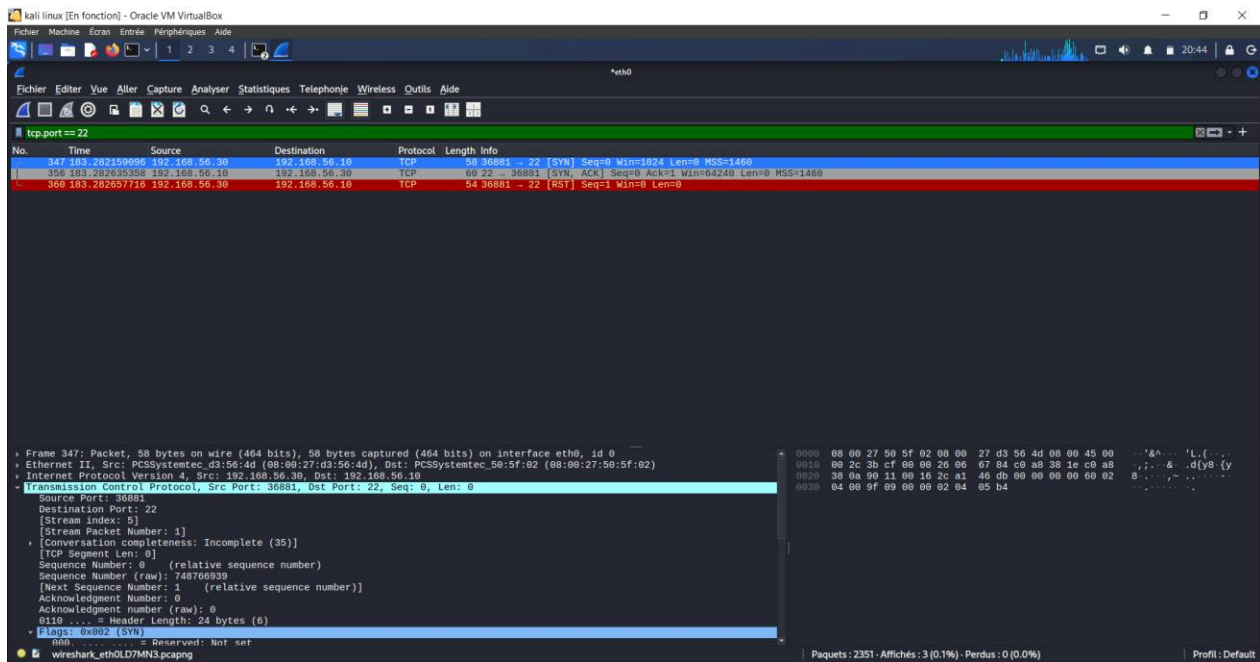
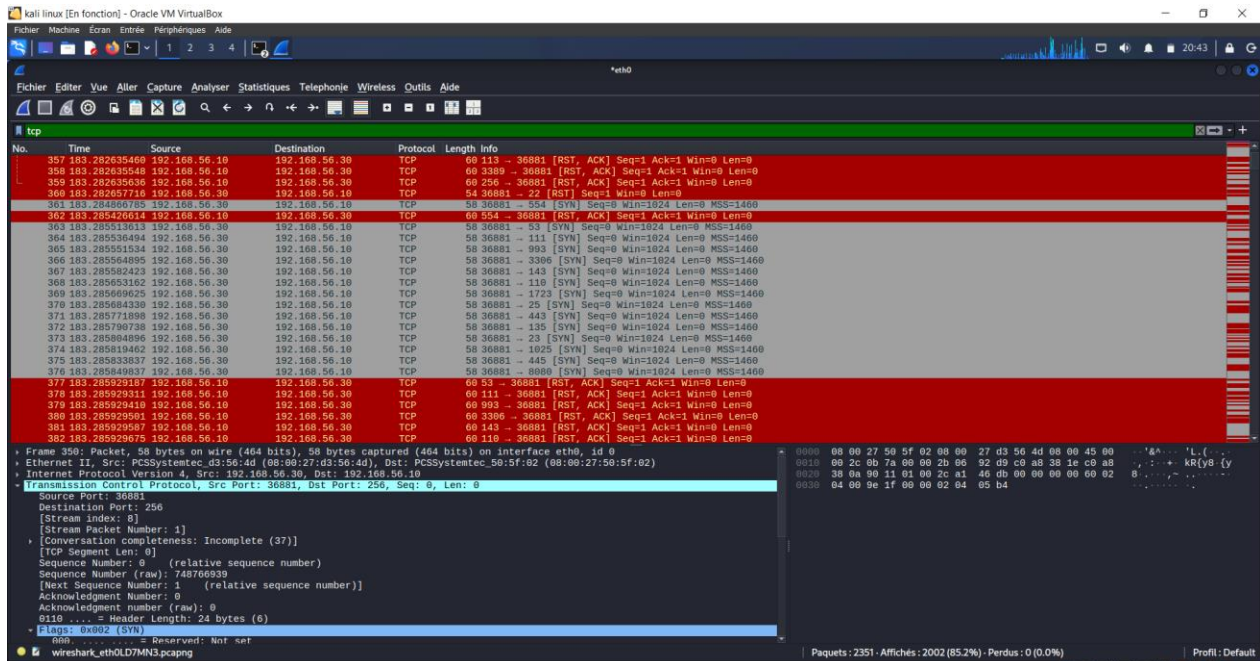
666 ... 5 Reserved: Not set

Wireshark eth0/DMZ.pcapng

Paquets: 2351 - Affichés: 2002 (85.2%) - Perdus: 0 (0.0%)

Profil: Default

Tcp.port==80



5. Lien avec la cybersécurité en entreprise

- **Utilisation en SOC :** Les analystes utilisent Wireshark pour enquêter sur des alertes. Ils ouvrent les fichiers de capture pour confirmer s'il s'agit d'une attaque réelle.

- **Détection d'attaque :** En surveillant les flux, on peut identifier des scans, des attaques par déni de service (DoS) ou des communications avec des serveurs de commande (C2).

6. Conclusion

Ce TP a permis de comprendre qu'une simple commande génère un bruit considérable sur le réseau. J'ai appris à manipuler les filtres pour extraire des preuves d'activité.

- **Difficultés :** Identification de l'interface réseau active.
- **Limites de Wireshark :** Volume de données important et difficulté de lecture du trafic chiffré sans clé.